

Aufgaben zum Aufgabenblock 4 - C++-Teil für TI und ITSEC

1.0 Basics

1.1 Datentypen

Schreibe ein Programm, mit welchem du eine Ganzzahl, eine Fließkomma-Zahl und einen

Text einliest und formatiert ausgiebst:

```
Gib einen Namen ein:
Gib eine Matrikel-Nummer ein:
Gib die ETCs ein:

-----

Name           : Max Mustermann
Matrikel-Nummer: 123456
ETCs           : 33.5
```

1.2 Entscheidungen

Schreibe ein Programm, welches eine Konstante beinhaltet, in der du die Nummer

Pi speicherst. Anschliessend sollst du ein Programm schreiben, welches:

Anforderungen:

- eine Zahl einliest (Radius)
- und dann die Fläche des Kreises berechnet
- anschliessend soll ein zweiter Radius eingegeben werden
- am ende soll ausgegeben werden, ob Kreis 1 oder Kreis 2 grösser war

```
Radius 1: 5
Radius 2: 2

Flaeche Kreis 1: 78.54
Flaeche Kreis 2: 12.57

⇒ Kreis 1 ist grösser
```

1.3 Loops, Arrays und Funktionen

Ziel ist es, ein kleines Menu zu schreiben, welches die Menu-Punkte so lange ausgibt, bis der Nutzer eine bestimmte Eingabe gemacht hat. Kapsel die Einzel-Teile in separate Funktionen (zum Beispiem print_main_menu, usw.). Arbeite zum Speichern der Menu-Punkte mit Arrays und uebergebe diese per Referenz.

```
Hauptmenu
1. Werte eintragen
2. Werte anzeigen
3. Werte bearbeiten
4. Werte loeschen
Zahl eingeben (0 zum Beenden):
>>>> 1
=> es wurde Werte eintragen ausgewaehlt

-----

Hauptmenu
1. Werte eintragen
2. Werte anzeigen
3. Werte bearbeiten
4. Werte loeschen
Zahl eingeben (0 zum Beenden):
>>>>
Programm beendet
```

1.4 Vektoren

Schreibe ein Programm, welches Eingaben in einem `std::vector` speichert. Die Eingabe soll bei 0 enden. Anschliessend sollen die Zahlen verarbeitet werden:

Anforderungen:

- größte Zahl ausgeben
- kleinste Zahl ausgeben
- Durchschnitt ausrechnen

1.5 Maps

Schreibe ein Programm, welches Artikel und ihre Artikel-Nummern speichert. Via Loop sollen wieder Eingaben getaetigt werden, bis mit einer leeren Eingabe die Eingabe-Funktion beendet wird. Anschliessend sollen Artikel-Nummern eingegeben

werden koennen. Ist die Artikel-Nummer vorhanden, soll der entsprechende Artikel ausgegeben werden, ist er nicht vorhanden, soll ein entsprechender Hinweis gegeben werden:

```
Artikel-Nr: 123456
Artikel-Name: Kaffee

-----

Artikel-Nr: 654321
Artikel-Name: Schokolade

-----

Artikel-Nr:

** Eingabe beendet **

= Suche gestartet =
---- mit 0 beenden ----
>>> 123456
=> Kaffee
.
.
.
```

2.0 Objektorientierung

Ziel ist es, ein kleines Programm zur Prüfungsanmeldung zu schreiben. Studierende sollen angemeldet, abgemeldet werden können. Zudem soll geprüft werden, ob ein Studierender bereits angemeldet ist, sowie soll eine Liste über alle Anmeldungen erstellt werden.

2.1 Einfache Klassen

Zunächst sollen die Basis-Klassen definiert werden. Schreibe jeweils sinnvolle Konstruktoren und teste die Klassen getrennt.

class Student

- definiere eine Klasse Student
- es soll mindestens folgende Attribute geben:
 - Name

- Geburtsdatum
- Matrikel-Nummer
- Studienfach
- Prüfungsordnung
- Semester

class Pruefungsamt

- definiere eine Klasse Pruefungsamt
- folgende Attribute:
 - pruefungen (map)
- folgende Methoden:
 - anmelden
 - abmelden
 - pruefen
 - ausgeben

Aufteilung

Lagere den Code in eine separate Header-Datei aus und schreibe ein einfaches main programm, welches die vorherigen Tests beinhaltet.

Menu

Schreibe in der main-Datei deines Programms eine Funktion namens `run()`.

Hier soll ein Menu definiert werden:

```
1: Anmelden
2: Abmelden
3: Anmeldungen anzeigen
4: Dateien einlesen
0: Ende
```

Schreibe ein switch-Statement, und gebe mit einfachen prints auf der Konsole Platzhalter aus. Der main-Loop soll bereits sauber beendet werden, wenn 0 als Eingabe erfolgt.

Daten-Erfassung

Beim An- und Abmelden müssen entsprechend Daten gesammelt werden. Schreibe

dazu entsprechend folgender Anforderungen in den entsprechenden Bereichen den Code:

Anmelden:

- es sollen alle Daten gesammelt werden, damit das Student Objekt sauber angelegt werden kann (siehe oben)
- baue Validierungen ein:
 - die Matrikel-Nummer sollte immer 6 Stellen haben
 - der Name darf keine Ziffern oder Sonderzeichen enthalten
- wenn alle Daten gesammelt wurden, soll ein Student Objekt erzeugt werden
- Optional kann der Eingabe-Prozess so lange weitergefuehrt werden, bis er explizit ueber eine Eingabe abgebrochen wird. Speichere die entstehenden Student-Objekte sinnvoll
- Optinoal: schreibe alles in Funktionen

Abmelden

- es soll einfach nach einer Matrikel-Nummer gefragt werden
 - validiere sie mit der obigen Validierungsanforderung
- schreibe weitere Moeglichkeiten wie nach Name usw. und speichere diese zwischen

Methoden Pruefungsamt

Implementiere nun die Methoden der class Pruefungsamt aus. Schreibe zuerst die pruefen-Methode, da diese auch beim Anmelden und Abmelden verwendet werden

soll und mit der verhindert werden soll, dass zweimal der gleiche Studierende angelegt wird.

Main Programm

Baue nun alles zusammen. Bei Anmelden und Abmelden muss nun noch mit dem Objekt Pruefungsamt interagiert werden und die entsprechenden Prozesse hinzugefuegt werden. Bei anmelden soll dann ein Student Objekt gespeichert werden (in der internen Map). Bei abmelden entsprechend mit dieser interagiert werden. Schreibe wenn noetig Hilfsmethoden fuer das Pruefungsamt (filter, search, usw.)

Daten einlesen

Schreibe nun eine Funktion, welche einen String uebergeben bekommt, welcher ein Dateinamen raepresentiert. Die Funktion soll die uebergebene Datei einlesen und anschliessend eine Liste bzw. einen Vektor mit Student Objekten erzeugen aus der Datei. Schreibe eventuell Hilfsfunktionen und ueberpruefe beim Einlesen, ob ein Student-Objekt korrekt erzeugt werden kann.

Finish

Fuege diese Funktion nun deinem Menu in der main-Datei hinzu und bringe alles zum Laufen. Teste das Programm ausgiebig und versuche eigene Dateien zu erstellen und einzulesen.